

A Quantitative Analysis of the Real-time Capabilities of Linux with PREEMPT_RT

This paper investigates the real-time capabilities of Linux, both for the vanilla kernel and for kernels with a real-time patch (PREEMPT_RT) applied. Scheduling latency is authors' parameter of interest and it is measured across various load conditions for both the kernels. Their experiments suggest that a real-time patch can improve the deterministic behaviour of Linux, thereby offering an alternative to proprietary, hard real-time operating systems.

The distinguishing characteristic of real-time systems is their ability to respond to events in a timely fashion, i.e., they are deadline oriented. Further, depending on how important deadlines are to the system, real-time operating systems are classified into hard and soft real-time. In the case of the latter, these systems provide a 'best-effort' service; they aim to complete tasks within the deadline, but deadline misses result in performance degradation and not a system failure. For instance, an ATM machine may not be able to meet a transaction deadline and may timeout, which is acceptable. Hard real-time systems, on the other hand, have imperative deadlines that they cannot afford to miss without causing serious damage to the system. An example would be an airbag inflation system in an automobile. The airbag must inflate within a few milliseconds of an accident; else it could be fatal to the driver.

Some of the popular real-time operating systems currently being used are VxWorks, QNX, eCOS and FreeRTOS. Even though Linux is designed as a GPOS (general-purpose operating system), the Linux kernel also has the features to customise the required options for use in embedded systems. The vanilla Linux kernel supports soft real-time capabilities but not hard real-time features. However, since Linux is the most prevalent operating system used by developers of embedded systems, this paper aims to inspect and analyse the real-time capabilities of Linux and look at ways to further augment these capabilities. Early Linux (1.x) had no facility for kernel pre-emption.

Once a task started executing in kernel mode, it could not be pre-empted and had to voluntarily give up kernel access. Linux 2.x introduced SMP and the Big Kernel Lock. Linux 2.6 was the first kernel to introduce pre-emption. However, spinlock sections were not preemptible. 2.6 comes with configuration options: CONFIG_PREEMPT_VOLUNTARY and CONFIG_PREEMPT. Despite using the CONFIG_PREEMPT option, critical sections protected by spinlocks are not pre-emptible. As a result, worst case latencies are still significantly high and not suitable for the strict deadlines of hard real-time systems. Therefore, to use Linux as a hard, real-time OS, modifications need to be made to the kernel.

There are two different approaches to providing a real-time performance with Linux:

1. Improving the Linux kernel pre-empt ability.
2. Adding a new software layer beneath the Linux kernel with full control of interrupts and processor key features.

